3. Datentypen in Powershell

Das Cmdlet *Get-Member* zeigt zu jedem beliebigen Objekt die Vorlagedefinition an.

```
$Text = "Das ist ein Text"
Get-Member -InputObject $Text
```

Die Ausgabe von *Get-Member* listet den Namen des Typs und alle wissenswerten Informationen auf.

Die einfachsten Typen, denen man in Powershell begegnet, sind Ganzzahlen, auf Englisch Integer oder kurz *Int*. Weitere einfache Datentypen, die auch als **Werttypen** bezeichnet werden, sind:

- *Double*: Zahlen mit Nachkommastellen
- *Bool*: die Wahrheitswerte true (wahr) und false (falsch)
- *Char*: ein einzelner Character oder Buchstabe

Außerdem gibt es komplexe Typen oder **Verweistypen**, die sich aus mehreren einfachen Datentypen zusammensetzen. Davon gibt es eine riesige Anzahl. Im Folgenden ein paar Typen, mit denen man es in Powershell oft zu tun hat:

- *String*: ein Text, der sich aus einer Verkettung von Chars zusammensetzt
- *Arrays*
- *Hashtable*
- *Datetime*: Datum und Uhrzeit, in einem Objekt gemeinsam gespeichert
- *FileSystemInfo*: speichert Daten über Dateien
- *Process*: speichert Informationen über Prozesse

Komplexe Typen werden von Cmdlets generiert, bis auf Strings. Um zum Beispiel ein Datetime-Objekt mit dem aktuellen Datum zu generieren, kann *Get-Date* verwendet werden.

```
Get-Date
```

Get-Date und Get-Member

Get-Date stellt aber auch eine Reihe von Parametern zur Verfügung, um beliebige Daten zu generieren.

```
Get-Date -Year 2023 -Month 12 -Day 24
```

Dass es sich wirklich um einen Datetime-Typen handelt, wird wieder mit dem Cmdlet *Get-Member* überprüft.

```
$Date = Get-Date -Year 2023 -Month 12 -Day 24
Get-Member -InputObject $Date
```

Hier eine abgekürzte Ausgabe des Kommandos:

```
 TypeName: System.DateTime
Name                 MemberType     Definition
----                 ----------     ----------
```

```
Add                  Method        datetime Add(timespan value)
AddDays              Method        datetime AddDays(double value)
AddHours             Method        datetime AddHours(double value)
AddMilliseconds      Method        datetime AddMilliseconds(double value)
AddMinutes           Method        datetime AddMinutes(double value)
AddMonths            Method        datetime AddMonths(int months)
AddSeconds           Method        datetime AddSeconds(double value)
AddTicks             Method        datetime AddTicks(long value)
[•]
Date                 Property      datetime Date {get;}
Day                  Property      int Day {get;}
DayOfWeek            Property      System.DayOfWeek DayOfWeek {get;}
DayOfYear            Property      int DayOfYear {get;}
Hour                 Property      int Hour {get;}
```

Die erste Zeile gibt den Namen des Typs an, hier `System.DateTime`. Typen werden in ordnerähnlichen Strukturen abgelegt, genau wie Dateien. Der Oberbegriff hierfür ist Namespace oder Namensraum.

Namespaces findet man überall. Das Dateisystem definiert einen Namespace genauso wie Internetnamen.

Die Ordner werden nicht mit Backslashes voneinander getrennt, wie im Dateisystem, sondern mit Punkten. `System.Datetime` bedeutet also nichts anderes, als dass die Vorlage sich im Ordner `System` befindet. `System` ist ein Ordner erster Ordnung, also ganz oben in der Hierarchie. Und natürlich gibt es auch für Typen Unterordner.

Objekte haben Eigenschaften, das wissen wir bereits aus der vorigen Übung. Die **Eigenschaften**, die im Typ festgelegt sind, werden in der Ausgabe von *Get-Member* in der Spalte Membertype als **Property** angezeigt. In der Spalte Definition kann man sehen, welchen Datentyp die Eigenschaft selbst hat. Das ist oft ein einfacher Datentyp wie ein INT, kann aber auch ein komplexer Datentyp sein.

Außerdem ist hier noch der MemberType *Method* zu finden. Methods oder Methoden sind Programme, die vom Programmierer des Typs mitgeliefert wurden und die meistens den **Inhalt eines Objekts manipulieren**.

Zum Beispiel kann die Methode *AddDays* verwendet werden, um auf einen bestehenden Datumswert Tage zu addieren oder zu subtrahieren, indem man einen negativen Wert addiert.

Methoden in Powershell

Methoden rufen sich wie Eigenschaften auf, allerdings mit **runden Klammern** hinter dem Namen. Die funktionieren wie Cmdlets, stammen aber aus dem **.net-Framework**, auf dem Powershell basiert. Deswegen findet die Übergabe von Parametern bei Methoden nicht mit Parametername und Argument statt. Stattdessen ist die Reihenfolge der Parameter festgelegt, man übergibt nur die Argumente – aber immer in der richtigen Reihenfolge! (Vergleiche die Programmierung in Java!)

```
$Jetzt = Get-Date
$Jetzt.AddDays(10)
$Jetzt.AddDays(-10)
```

Betrachtet man das Ergebnis, stellt man fest, dass die Methode das Objekt nicht verändert hat. `$Jetzt.Addays(10)` gibt den Inhalt von `$Jetzt +10` Tage aus, doch wenn man anschließend `-10` Tage addiert, kommt nicht wieder das aktuelle Datum heraus, sondern das Datum vor 10 Tagen.

Der Inhalt der Variablen, also das Objekt, wird durch die Methode nicht verändert. Stattdessen wird ein neues Objekt ausgegeben. Um das Objekt zu verändern, muss das neue Objekt in der alten Variablen gespeichert werden.

```
$Jetzt = $Jetzt.AddDays(-10)
```

Nur: Der Variablenname ergibt jetzt keinen Sinn mehr, das kann verwirren. So etwas sollte vermieden werden, zum Beispiel, indem der neue Wert in eine passende neue Variable gespeichert wird.

```
$Damals = $Jetzt.AddDays(-10)
```

Da Methoden nicht von Powershell stammen, sind sie nicht durch *Get-Help* dokumentiert. Es gibt aber einen Trick, sich die möglichen Parameter-Übergaben anzeigen zu lassen. Hierfür einfach die Methode wie eine Eigenschaft aufrufen – ohne die runden Klammern hinter dem Methodennamen.

```
$Jetzt.AddDays
```

Als Ergebnis werden die Überladungen (Overloads) ausgegeben. Der Begriff klingt kompliziert, ist jedoch nur ein verwirrend klingender Name, um die verschiedenen Parameteraufrufe zu benennen.

```
OverloadDefinitions
-------------------
datetime AddDays(double value)
```

Die Fähigkeiten von `AddDays()` sind überschaubar, es gibt nur einen Parameter und eine Überladung. In den runden Klammern sind alle Pflichtparameter mit Datentyp und dem Parameternamen zu finden. Die einzelnen Parameter sind durch Kommata getrennt. AddDays hat nur einen Parameter namens *value*, der als Datentyp *Double* erwartet, eine Zahl mit oder ohne Kommata.

Wer bloß die Methoden oder Eigenschaften eines Typs sehen möchte, kann mit dem Parameter *-Membertype* die Ausgabe filtern.

```
Get-Member -MemberType Method -InputObjekt $Jetzt
Get-Member -MemberType Property -InputObjekt $Jetzt
```

Strings und Wertetypen kann man in Powershell ohne ein Cmdlet erzeugen, indem man sie direkt in die Kommandozeile eingibt, sie werden dann direkt wieder in die Konsole zurückgeschrieben. Damit Powershell Strings von Befehlen unterscheiden kann, müssen Strings in einfache oder doppelte Anführungszeichen eingefasst werden. Sollen die Werte weiterverwendet werden, weist man sie einer Variablen zu.

```
'Viele Grüße aus der Kommandozeile'
$Ganzzahl = 1
$Kommazahl = 1.2
$Name = 'Holger'
$Begruessung = "Hallo $Name"
$Begruessung2 = 'Hallo $Name'
$Begruessung
$Begruessung2
```

Einfache Anführungszeichen sind sogenannte Literale oder buchstäbliche Strings, sie werden genauso ausgegeben, wie sie eingegeben wurden. Strings in doppelten Anführungszeichen werden als erweiterbare (expandable) Strings bezeichnet, weil Powershell sie vor der Ausgabe noch einmal auf Steuerzeichen und Variablen prüft und diese gegebenenfalls ersetzt. Im Beispiel oben ergibt die Ausgabe von `$Begruessung` und `$Begruessung2` also unterschiedliche Ergebnisse.

Nun zur Aufgabe dieses Teils:

- **Ü3.1.1** Gib den Text "Bitte geben Sie eine Zahl ein: " in die Konsole aus.

```
Write-Host "Bitte geben Sie eine Zahl ein: "
```

- **Ü3.1.2** Speichere das Ergebnis der Berechnung 2 + 3,1 in einer Variablen und lass dir den Datentyp des Ergebnisses anzeigen.

```
$sum = 2+3.1
Get-Member -InputObject $sum
```

- **Ü3.1.3** Es gibt neben dem Cmdlet *Get-Member* noch eine Methode, die dir von jedem Objekt zur Verfügung gestellt wird und den Datentyp des Objekts ausgibt. Wie heißt diese Methode? Tipp: *Get-Member* zeigt dir alle Methoden eines Objekts an. Teste sie und schau dir die Unterschiede zu *Get-Member* an.

```
.GetType()
```

- **Ü3.1.4** Du kannst mit einer Methode, die für jeden Typ definiert ist, jedes Objekt als Text anzeigen lassen. Wie heißt diese Methode? Probiere sie an einem Datumsobjekt aus.

```
.toString()
```

- **Ü3.1.5** Von welchem Typ ist ein Ordner abgeleitet? Speichere mit *Get-Item* einen Ordner in einer Variablen und ermittle den Typ.

```
$dir = get-Item ./Transcripts
$dir.getType()
```

- **Ü3.1.6** In welchem Namespace befinden sich die Prozess-Objekte, die *Get-Process* zurückliefert?

```
$Process = Get-Process
$Process[0].GetType().Namespace
```

- **Ü3.1.7** Strings verfügen über eine Reihe von hilfreichen Methoden, die den String manipulieren können. Versuche mit Hilfe der String-Methoden folgende Probleme zu lösen:
  1. Trenne die Systemvariable *Path*, die in einem langen String mehrere mit Semikolon getrennte Ordnernamen enthält, in ihre einzelnen Elemente auf.

     ```
     $Path = $env:path
     $Path = $Path.Split(';')
     ```

  2. Ersetze in der *Path*-Variable alle `\` durch ein `/`.

     ```
     $Path= $Path.Replace('\', '/')
     ```

  3. Entferne alle Doppelpunkte aus der *Path*-Variable. Kleiner Tipp: Manchmal kann man mit Ersetzen auch löschen.

     ```
     $Path = $Path.Replace(':',
     '')
     ```

Datentypen konvertieren

Einfache Datentypen können kombiniert werden. Hierfür verwendest du Operatoren wie +, –, * (Multiplikation) oder / (Division). Allerdings unterstützen nicht alle Datentypen alle Operatoren. Strings kann man zum Beispiel nicht miteinander dividieren, aber Zahlen.

```
3 + 2
2 * 10
10/2
"Hund " + "Katze"
```

Die Operatoren legen fest, wie Powershell zwei Objekte kombinieren soll. Die Kombination ist immer durch den Datentyp festgelegt, man kann immer nur zwei gleiche Datentypen kombinieren. Bei Zahlen ist das eine mathematische Operation, bei Strings werden die Buchstaben aneinandergehängt. Das Ergebnis von "Hund " + "Katze" ist also "Hund Katze". Was aber passiert dann hier?

```
10 * "10"
"*" * 10
```

In beiden Fällen spuckt uns Powershell ein Ergebnis aus, nämlich `100` beziehungsweise `**********`. Powershell hat offensichtlich im ersten Fall den Text "10" als Zahl interpretiert und im zweiten Fall den String multipliziert.

Diese beiden Operationen sind explizit in Powershell eingebaut worden, um einem das Leben einfacher zu machen. Im ersten Fall wandelt Powershell den String "10" vor der Multiplikation in eine Zahl um, im zweiten Fall wird der String 10-mal hintereinander ausgegeben. In allen anderen Fällen kann man nur Objekte des gleichen Datentyps kombinieren. Dazu folgendes Beispiel:

```
$Zahl1 = 1
$Zahl2 = 2
$Zahl3 = "3"
$Zahl4 = "vier"
$Zahl1 + $Zahl2
$Zahl1 + $Zahl3
$Zahl3 + $Zahl1
$Zahl1 + $Zahl4
```

Offenbar sind in dem Beispiel verschiedene Datentypen kombiniert worden, aber Powershell hat nur einmal eine Fehlermeldung ausgegeben, und zwar beim letzten Beispiel.

```
Der Wert "vier" kann nicht in den Typ "System.Int32" konvertiert werden. Fehler: "Die
Eingabezeichenfolge hat das falsche Format."
```

`$Zahl1 + $Zahl3` sollte das korrekte Ergebnis `4` ausgegeben haben, während `$Zahl3 + $Zahl1` das eindeutig falsche Ergebnis `31` auswirft. Was ist passiert?

Das Problem mit unterschiedlichen Datentypen

Datentypen umwandeln

Unterschiedliche Datentypen können nicht kombiniert werden. Manchmal lassen sich Datentypen jedoch umwandeln. Ausnahmslos alle Typen lassen sich in einen String konvertieren. Dafür sorgt die Standardmethode *ToString()*, über die alle Objekte verfügen.

Auch Strings, die aus Ziffern bestehen, lassen sich in Zahlen umwandeln. Um Administratoren mit solchen Details nicht zu belästigen – Powershell soll möglichst einfach anzuwenden sein –, versucht Powershell, Konvertierungen selbstständig durchzuführen und verwendet eine einfache Regel: Der zweite Wert einer Operation wird immer in den Datentyp des ersten Werts umgewandelt.

Deswegen meldet Powershell einen Fehler, wenn man `$Zahl1` mit `$Zahl4` addiert – das Wort vier lässt sich nicht in eine Zahl konvertieren. Wenn der Datentyp des ersten Werts aber ein String ist, wird der zweite Wert auch in einen String konvertiert, und `+` verknüpft beide zu einem neuen.

Besser ist es, den Datentypen immer manuell zu konvertieren. Das ist nicht schwierig, man schreibt einfach den gewünschten Datentyp in eckige Klammern vor den zu konvertierenden Wert.

```
$ZahlDouble = [Double]1
$ZahlString = [String]10
$ZahlInt = [Int]"100"
$Datum = [DateTime]"2023-11-11"
$Pfad = [String](Get-Item -Path C:\Windows\Notepad.exe)
```

Dazu eine kleine Aufgabe:

- **Ü3.2.1** Führe die oben genannten Beispiele aus und prüfe den Datentyp der einzelnen Variablen.

```
Get-Member -InputObject $ZahlDouble # System.Double
Get-Member -InputObject $ZahlString # System.String
Get-Member -InputObject $ZahlInt # System.Int32
Get-Member -InputObject $Datum # System.DateTime
Get-Member -InputObject $Pfad # System.String3
```

Wie am Datumsbeispiel zu sehen ist, klappt die Konvertierung nicht nur mit einfachen, sondern auch mit komplexen Datentypen. Voraussetzung ist, dass der Programmierer der Klasse eine Konvertierung in den entsprechenden Zieldatentyp vorgesehen hat.

Konvertierungen in Strings funktionieren immer, da jedes Objekt über eine Methode *ToString()* verfügt, die immer den Datentypen zurückgibt – soweit der Programmierer keine eigene Idee hatte, wie der Text aussehen soll.

Datentypen festlegen

Eine Variable kann bei der Deklaration auch auf einen Datentyp festgelegt werden. Powershell versucht dann bei jeder Zuweisung, den zugewiesenen Wert in den Datentyp der Variable zu konvertieren.

Funktioniert das nicht, meldet Powershell einen Fehler und weigert sich, die Variable zuzuweisen. Eine typisierte Variable kann nur einen anderen Datentyp annehmen, wenn sie vorher gelöscht oder neu deklariert wird. Zum Festlegen des Datentyps muss der Datentyp bei der Deklaration einfach vor die Variable statt vor den zuzuweisenden Wert geschrieben werden.

```
[Double]$ZahlDouble = 1
[String]$ZahlString = [String]10
[Int]$ZahlInt = "100"
[DateTime]$Datum = "2023-11-11"
[String]$Pfad = Get-Item -Path C:\Windows\Notepad.exe
```

- **Ü3.3.1** Deklariere eine Variable als Int und weise ihr zuerst den String "1" und anschließend den Wert "eins" zu. Versuche, das Ergebnis zu erklären.

  ```
  [int]$ZahlInt = "1"
  $ZahlInt = "eins"
  MetadataError: Cannot convert value "eins" to type "System.Int32". Error: "The input string 'eins' was not in a correct format."
  ```

- **Ü3.3.2** Ändere die Deklaration so, dass die Zuweisung der Zahl "eins" funktioniert.

  ```
  [string]$ZahlInt
  ```

- **Ü3.3.3** Speichere das aktuelle Datum in einer als String deklarieren Variable und prüfe den Datentyp.

  ```
  [String]$Date = Get-Date
  $Date.Get-Type()
  ```